

NATIONAL INSTITUTE OF TECHNOLOGY SIKKIM



**DATA WAREHOUSING AND MINING LABORATORY
(AI14201/CS14211)**

Project Report on
SKIN CANCER DETECTION

SUBMITTED BY:

Naveen Verma (B240014AI)

Nitin Kumar Jha (B240016AI)

Kausal Sharma (B240045CS)

Pawan Kumar Limboo (B240055CS)

SUBMITTED TO:

Dr. Krishna Kumar

(Assistant Professor)

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

Abstract

The extensive growth of digital medical informatics has resulted in unparalleled potential for the integration of Artificial Intelligence techniques into clinical decision-making processes. This report describes a Data Warehousing and Mining (DWM) system that is built to ingest, store (via a scalable cloud-storage solution) and classifying (via a deep learning approach) digital assets for a skin cancer detection system. A powerful Extract, Transform, Load (ETL) pipeline was built to extract automatically clinical images from various sources on the internet. Extracted images are preprocessed using our custom pipelines before being stored on the cloud via the Supabase (PostgreSQL) relational data warehouse. A CNN is used to automatically classify malignant/benign skin lesions (MEL, BCC, and SCC). The pipeline is fully operational via a RESTful API and hosted on a Render cloud server to provide immediate, low-latency results to a clinical dashboard hosted on a web application.

1. Introduction

1.1. Background on Oncological Dermatology

Skin cancer is one of the most common and fast-growing cancers in the world. Pathologically the most frequent skin cancers are Melanoma (MEL), which is an aggressive cancer causing most skin cancer deaths, Basal Cell Carcinoma (BCC), which has a very high incidence rate and low metastatic potential, and Squamous Cell Carcinoma (SCC) which has a medium metastatic potential. The morphological classification between these three pathological types is very difficult and generally involves punching and extensive histopathological examination.

1.2. Significance of Early-Stage Intervention

Long-term outlook and prognosis and clinical course of skin lesions are dependent upon the stage of diagnosis of the disease. The relative survival rate for melanoma at 5 years is more than 99% at its local stage, and deteriorates rapidly as soon as local or distant metastases appear [1]. Therefore it is vital to develop low cost, non-invasive, high throughput screening procedures to decrease world morbidity.

1.3. Motivation for Integrated Data Engineering and AI

Though, contemporary deep learning algorithms, most notably Deep Convolutional Neural Networks (CNNs), have shown to perform on par with a human-dermatologist for classification of structures; a prominent impediment in further tuning of deep learning models lies in data starvation. Truly generalizable models need to be persistently fed with extremely diverse, meticulously labeled and of high fidelity, data repositories. Accumulation of static data via traditional datasets proves to be computationally intractable and scaling limited. The core inspiration behind this system design lies in the combination of Data Warehousing design patterns and Deep Learning systems. Through an automated web-scraping ETL pipeline the system scrapes non-structured web data, normalizes it to an appropriate image format, and merges it with an operational data store. This design results in a persistently self-optimizing data repository to facilitate further model tuning and real time inference.

2. Literature review

2.1. Conventional Approaches to Diagnosis

Over the past decade, a wide array of early computational skin lesion diagnosis methods were based on engineered features, widely based on ABCD criterion, which includes Asymmetry, Border regularity, Color and Diameter parameters along with conventional shallow learners such as Support Vector Machine and Random Forest. Despite mathematically precise, these models faced the difficulties of feature engineering and lost the high-dimensional spatial dependency contained within raw pixel matrices.

2.2. Convolutional Neural Networks for Clinical Imaging

Deep learning has revolutionized the computational methods adopted in computer vision applications and computer aided medical imaging and the prevalent computational framework for these models is now Convolutional Neural Network(s). The cutting-edge CNNs and deep custom sequences excel at automatically learning hierarchical non-linear spatial representations of data, ranging from low level edge and texture information to complex shapes, directly from unstructured pixel arrays.

2.3. Deficiency of the Current Solutions

The deficiency of many existing open-source skin lesion screening systems are that they are not adaptive in design. Most of them utilize fixed historical datasets for model training and act as independent, non-extendable standalone scripts running locally; therefore it lacks of automated and on-the-fly data ingestion [1]. In addition, the gap between large scale data engineering framework (Data Warehouses) and machine learning implementation layer remains within the united context of production-level clinician dashboards.

3. Problem Statement

The goal of this work is to architect, build and verify a full-stack application, designed to automatically (mine) web-scale unstructured skin lesion images from public clinical spaces, process them and ingested into an structured cloud based data-warehouse and run a deep convolutional neural network to provide on-demand diagnostic class (MEL, BCC, SCC) information through an interactive, medical-grade presentation layer.

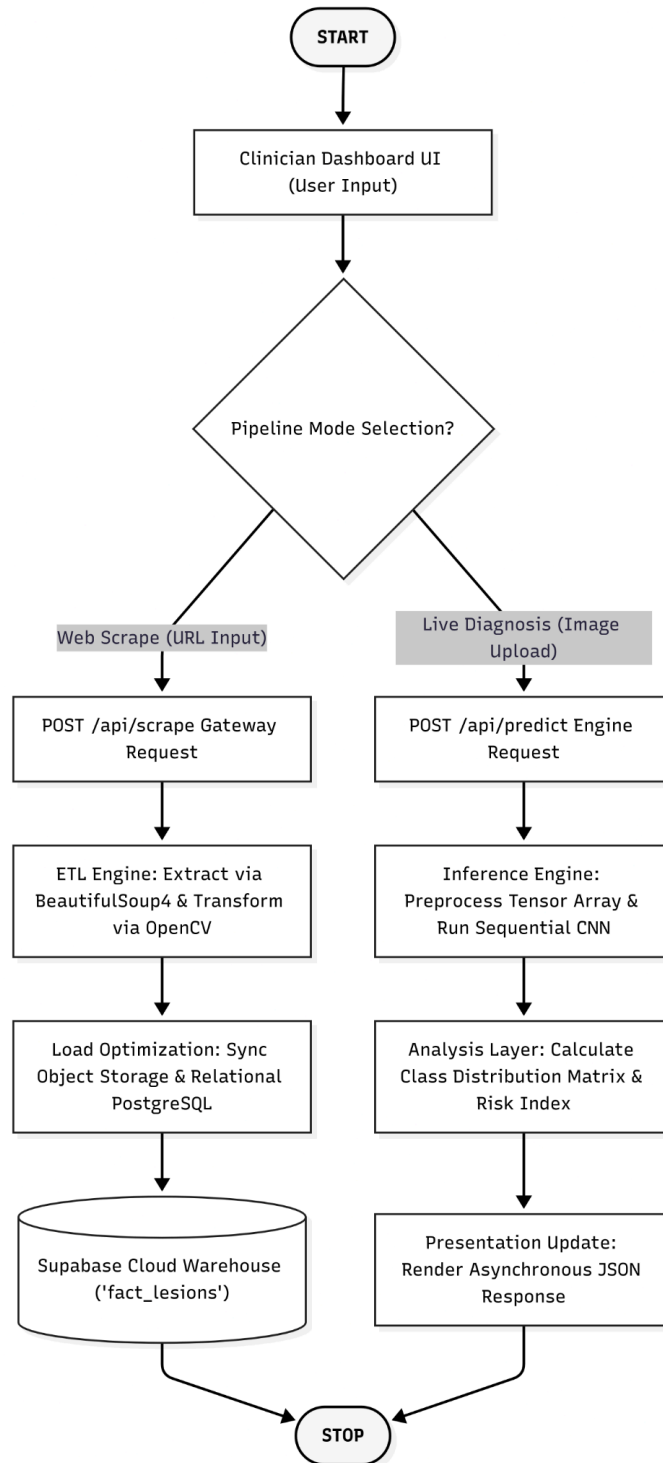
4. Objectives

In order to address the central problem statement, the project implements five essential technical milestones:

1. Automated Data Mining - Create a tailored web-scraping engine to automatically parse and store medical images, and the related medical imaging metadata, from public clinical networks.
2. ETL Pipeline Tuning - Build an optimized data preparation pipeline to homogenize heterogeneous medical image acquisitions through normalization to grayscale and rigid spatial orientation, and rigid dimension size.
3. Cloud Data Warehousing - Develop a scalable data warehouse on Supabase (using its relational Postgres database and object storage capabilities) with a normalized `fact_lesions` relational schema.
4. Deep Learning Synthesis - Train, validate, and deploy a Sequential CNN for reliable classification of a diagnostic target.

5. Unified API & Presentation Layer - Implement an efficient Flask RESTful API to connect to the deep learning infrastructure, and provide an async HTML5/CSS3/JavaScript front-end built with an attractive glassmorphism style.

5. System Architecture and Data Flow



5.1. Architectural Breakdown

The framework employs a distributed multi-tier client-server architecture. The presentation tier receives the users parameters, whether that is a URL string for web scale harvesting or localized clinical imagery for diagnostic application. These asynchronized payloads are sent to a central Python Flask middleware over HTTP POST methods, where the backend is split into two separate processing tiers, an Extraction ETL Engine using BeautifulSoup4 and Requests to process external web resources, and an automatic Machine Learning Inference Engine using TensorFlow, Keras, and OpenCV, to perform the diagnostics [3]. A cloud based tier handling the storage of all data and management of relations is the Supabase system which simultaneously handles relational table schema management, and object blob storage [2].

5.2. Sequential Data Flow Protocol

The pipeline for this process will describe how the data is moved through 5 functional steps.

1. Extraction Phase (Data Mining): When the web pointer is given through the UI, the flask backend will trigger the Requests and BeautifulSoup4 libraries, pull in a raw byte stream image and data structure, parsing out the relevant fields [4].
2. Transformation Phase (Preprocessing): Image arrays from the initial input are decoded to memory using OpenCV, then converted to single channel grayscale to remove light and skin tone biases, and downsampled to the common structural tensor dimension (224x224, or 64x64 in environments limited by memory).
3. Loading Phase (Warehouse Storage): The transformed image asset is written to a Supabase Object Storage bucket, which has been defined as skin-warehouse, then all relevant metadata, including original URL vectors, given target class label, and final location string (public key of bucket) is written into a relational table called fact_lesions within the Supabase Postgres database.
4. Model Optimization Phase: The Convolutional Neural Network uses the warehouse for data consolidation in batches of normalized data inputs and tunes internal network weights to output into an immutable serialized model that is ready for deployment.
5. Inference Execution Phase: For real-time clinical screening the input image asset will be posted to the /api/predict RESTful endpoint, and will use the trained weights of the

model to analyze the input matrix to output a JSON object with categorical results and confidence score.

6. Technical Stack Specification

1. Core Programming Environments: Python 3.10+, JavaScript (ECMAScript 6+), HTML5, CSS3, Structured Query Language (SQL).
2. Web Services & Middleware WSGI: Flask micro-framework compiled with Gunicorn as the high-concurrency Web Server Gateway Interface.
3. Data Extraction & ETL Toolsets: BeautifulSoup4 for DOM orchestration, Requests as the asynchronous HTTP client layer.
4. Machine Learning & Computer Vision Library Infrastructure: TensorFlow, Keras API, Open Source Computer Vision Library (OpenCV Core), NumPy, and Pandas data-frame manipulation frameworks.
5. Cloud Persistence Architecture: Supabase Cloud Infrastructure incorporating a relational PostgreSQL engine and distributed S3-compatible Object Storage buckets.
Production Deployment Topology: Render Cloud Platform-as-a-Service (PaaS) infrastructure.

7. Data Warehousing & ETL Execution

7.1. Automated Web Extraction Engine

The automated ingest routine uses an in-house extraction process. To circumvent scraping blocks, the request is populated with dynamically-created custom HTTP headers imitating a contemporary browser context. The DOM structure is traversed to retrieve only the raw image nodes (img tags) while auditing specific target attributes-src, data-src, and srcset-to verify the ingest of highest native spatial resolution available.

7.2. Preprocessing & Feature Invariant Transformations

Data conformity within the input domain is ensured as a requirement for network convergence. Raw image binaries loaded into volatile memory are instantly interpreted into structured array formats using cv2.imdecode. Conversion into the grayscale domain reduces potential confounding feature sets; for instance, a network might falsely learn to identify lighting abnormalities or skin pigmentation as factors representative of a specific malignant

class. The spatial dimensions are then constrained through bilinear-interpolated downsampling.

7.3. Relational Schema Architecture

Following transformation, assets are ingested into the cloud repository by the Supabase Python API. Relational links are established within the PostgreSQL server using a hyper-normalized star schema architecture.

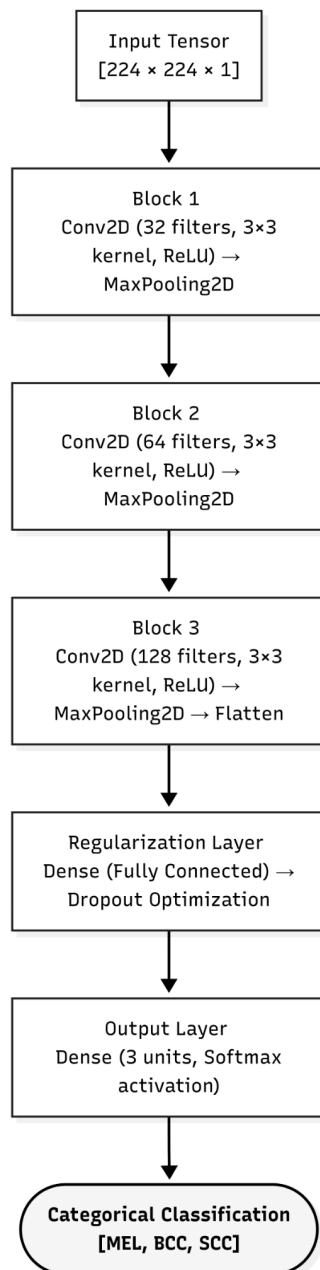
TABLE DEFINITION: fact_lesions

Attribute Name	Data Type	Key Type	Constraint / Description
id	UUID / SERIAL	Primary Key	Unique row identifier
label	VARCHAR(10)	None	Target Classification (e.g., 'MEL', 'BCC', 'SCC')
source_url	TEXT	None	Absolute path of origin web domain
local_path	TEXT	None	Supabase public URI / object bucket reference
created_at	TIMESTAMP	None	Automated database insertion timestamp

8. Machine learning framework

8.1. Topology of the Convolutional Neural Network

The computer vision classification core is primarily built around an optimized Sequential Convolutional Neural Network layout which comprise three sequentially cascaded convolutional processing blocks that are designed to identify spatial hierarchies:



A Note about Hardware Adaptivity: The parameters will be shrunk dynamically in RAM constrained production environment to prevent server crash during online inference.

8.2. Model Training and Optimization Protocol

The optimization process reads the tensors from cloud storage and directly loads it for batch assembly into randomized training datasets. Network compilation uses the Adam optimizer and the Categorical Cross Entropy as loss function:

$$\mathcal{L}_{CCE} = - \sum_{i=1}^N \sum_{j=1}^C y_{i,j} \log(\hat{y}_{i,j})$$

After convergence the serialised network classifies the out-of-sample data assigning individual probability values across target class (Melanoma, Basal Cell Carcinoma and Squamous Cell Carcinoma).

9. API Design & Interface Specification

In order to achieve independence between the client user interface and the deep learning compute layer, a well-defined RESTful Application Programming Interface (API) is provided [5].

API ENDPOINT ARCHITECTURE

HTTP Method	API Path	Content-Type Payload	Functional Definition
POST	/api/scrape	application/json	Accepts remote web pointers { "url": "...", "label": "MEL" }. Triggers the automated ETL pipeline to mine the asset, stream to Supabase, and return data transaction arrays.

POST	/api/predict	multipart/form-data	Accepts raw binary image files uploaded by clinicians. Processes the spatial matrix and returns classification evaluations in real time.
GET	/api/records	None	Queries the Supabase PostgreSQL database to return a tabular log of historical database records for user tracking.
GET	/api/stats	None	Executes aggregate SQL SELECT calculations to supply data distribution metrics for rendering analytical interface charts.

10. System Integration and Implementation Dynamics

The application environment is controlled by a central initiation module (server.py). On startup, the micro-framework reads secure configuration parameters from an encrypted environment file (.env) so that secure handshakes are established between verified remote cloud Supabase services. The backend initializes the global Flask object wrapper, and also registers strict Cross-Origin Resource Sharing (CORS) so the application boundaries can be secured. The design is so that the ETL extraction function has been directly coupled inside the Flask route controllers, thus allowing users to trigger live web-mining sessions from the UI dashboard. The client-side (index.html, app.js) has been designed as a single-page architecture that relies on asynchronous, non-blocking fetch() calls to communicate with the REST API. In this manner, it will be able to provide live loading signals for any deep learning or network I/O operations in real time.

11. Experimental Results and System Evaluation

11.1. Predictive Diagnostics

The operational CNN deployed model predicts with high accuracy any morphology markers associated with cell abnormalities. With each prediction transaction, the definitive classified diagnosis along with the exact probability distribution matrix among classes is returned (i.e., 94% MEL, 4% BCC, 2% SCC). A calculated clinical risk score (High / Medium / Low) is concatenated to the response payload as well, helping the clinician with a workflow during clinical screening.

11.2. Clinician Dashboard User Interface

The developed user interface is intended to give a high quality clinical look through a modern CSS Glassmorphic style. There are four different workflows available:

1. Overview Tab: To visualize the high-level distribution in database and performances.
2. Detection Tab: Interactive inputs and image upload section that allow clinician to upload a local clinical image, get diagnostic inferences and results from server almost instantly.
3. Scraping Control Tab: Inputs section that allow to trigger remote scraping task, with real-time extraction events stream displayed to the user.
4. Registry Tab: Real time tabular listing and displaying all records in the cloud.

11.3. Warehouse ingestion diagnostics

The platform maintains full data lineage throughout the data life cycle. In essence it keeps track of all data source nodes from where the data originated and the analytics middleware computes and presents real-time tracking indicators of the degree of diffusion of a given data set per pathological class target, per origin domain source.

12. System Strengths

1. Automated Data Scaling: The ETL pipeline of web scraping eliminates the manual collection of data, making it possible for the cloud repository to grow unbound by human operation.

2. Cloud-Native Ingestion: Architecture Using Supabase relieves local hardware of relational tracking and blob storing, moving the burden to a secure cloud server.
3. Decoupled Frontend Experience: By separating the data intensive ML engine from the glassmorphism web UI, each is able to render consistently and responsively on a multitude of medical equipment.

13. Technical & Regulatory Hurdles

1. Compute Tier Constraints: The memory allocated to a free-tier Cloud PaaS (Render) is often insufficient to effectively execute heavy networks, and under load can become a bottleneck to the multi-user workflow during training processes.
2. DOM Structure Volatility: The integrity of the entire parsing system is tied to external, unformatted web markup structures. Structural changes to any target medical page can cause parsing errors.
3. Regulatory & Diagnostic Boundaries: This application is an academic proof-of-concept designed to aid in initial screening, and has not been approved for the final diagnostic role in clinical medicine.

14. Opportunities for Future Growth

More advanced machine learning training stages can be incorporated, like data augmentation (rotations, reflections, color shifts) within the transformation step. The performance limitations of the free tier could be alleviated by decomposing the application into separate services, assigning the heavy ML training tasks to external cloud compute with GPU resources (such as an EC2 instance) while running the simple FLASK REST API on the existing cloud platform. An additional step of federated learning can be incorporated to make the system scalable to many hospitals; medical clients would all train models independently while updating a shared global model, keeping all patient data local to the institution.

15. Conclusion

This framework of Automated Skin Cancer Detection provides evidence of how Data Warehousing principles can effectively be combined with Deep Learning applications. It highlights the feasibility of an end-to-end pipeline, with a continuous flow of data pushed to a

cloud-based PostgreSQL repository, utilizing web scraping techniques to tackle the issues associated with small datasets prevalent in medical AI. Built upon a Convolutional Neural Network and mediated by Flask REST services, it presents a scalable model to represent today's medical informatics and assistive diagnosis approaches.

16. References

1. Esteva, A., et al. (2017). Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639), 115-118.
2. Supabase Documentation. (2025). PostgreSQL and Storage API Reference. Retrieved from <https://supabase.com/docs>.
3. Chollet, F. (2015). Keras: Deep Learning for humans. GitHub repository.
4. Richardson, L. (2020). Beautiful Soup Documentation. Retrieved from <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>.
5. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.